

Predator-Prey Gridworld

A Modular Multi-Agent Reinforcement Learning Testbed

Student Orientation • Week 1

Predator-Prey Archetype Gridworld Environment

Section 1: Why This Exists

The problem we solve and our mission

The Problem with Most MARL Environments

Most research environments tangle concerns that should stay independent:

- Environment dynamics and learning algorithm live in the same class
- Opaque reward computation — you cannot trace why a policy improves
- Global random state makes results non-reproducible across machines
- Changing one variable (e.g. reward shape) silently affects others
- Impossible to run clean ablations: observation vs reward vs algorithm

“We need an environment where each layer can be studied independently, with full reproducibility and zero surprise interactions between components.”

What This Project Is — and Is Not

IS

- ✓ A controlled lab for teaching and researching RL/MARL
- ✓ Config-driven — one YAML change = one experimental variable
- ✓ Fully deterministic — same seed, same trajectory, always
- ✓ Plugin-based — swap observations, rewards, algorithms
- ✓ Traceable — every transition verifiable by hand

IS NOT

- ✗ A high-performance training platform
- ✗ A continuous-action or visual environment
- ✗ A replacement for PettingZoo or SMAC
- ✗ Optimised for GPU-scale deep RL
- ✗ A ready-made publication benchmark

Section 2: Architecture

Four layers, one golden rule

The Four-Layer Architecture

Every component lives in exactly one layer. Dependencies flow downward only.

Layer 4 — Scripts & Orchestration

run_from_config.py • evaluate.py • sweep.py • render.py

Drives training runs from YAML config

Layer 3 — Baselines (Learning Algorithms)

IQL • CQL • MixedTrainer • (future: DQN, PPO, Nash Q)

Selects actions, updates value tables

Layer 2 — Plugins (Observations + Rewards)

5 observation builders • 3 reward functions

Transforms env state into percepts and signals

Layer 1 — Core Environment ★ IMMUTABLE ★

core/gridworld.py • core/agent.py

Grid physics, movement, capture — never modified

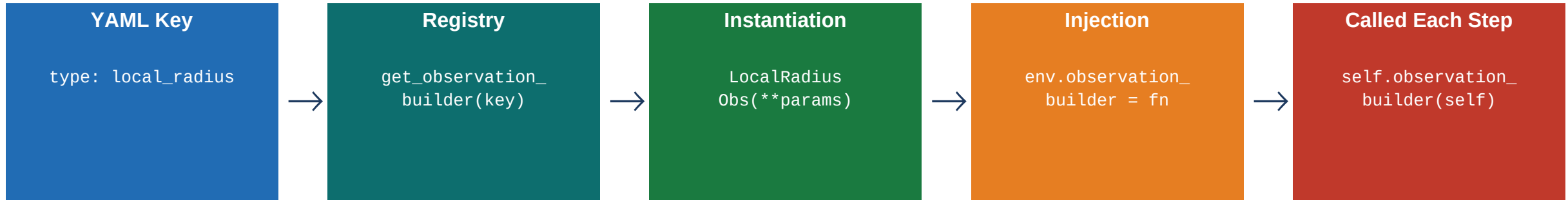
Five Core Design Invariants

These rules protect the scientific validity of every experiment run on this codebase.

Immutable Core	<code>src/core/</code> is never modified by contributors. Physics must not change between experiments.
Seeded Randomness	<code>env.reset(seed=n)</code> with the same <code>n</code> always produces the exact same trajectory, on any machine.
Read-Only Plugins	Observation builders and reward functions observe state — they never write it.
Config Drives Everything	Need a behavioral change? Write a plugin. If you think you need to edit core code, that is a design bug.
Black-Box Algorithms	Algorithms may only call <code>env.reset()</code> and <code>env.step()</code> . No reading of <code>env.agents</code> or internal state.

The Plugin & Registry Pattern

Plugins are discovered at runtime from YAML keys — no hardcoded imports in core.



Why this pattern?

- Adding a new observation builder: one new file + one registry line — zero core changes
- Algorithm author never imports the builder directly — YAML is the only coupling point
- Core never changes — all prior experiments remain valid after adding a plugin
- The pattern is identical across all three extension points: obs / reward / algorithm

Extension Points — The Three Contribution Pathways

Layer	Interface Method	Directory	Registry	YAML Key
Observation	ObservationBuilder.build(env) → dict	src/observations/	observation_registry.py	observation.type
Reward	RewardFunction.compute(env) → dict	src/rewards/	reward_registry.py	rewards[].name
Algorithm	BaseAlgorithm.select_actions() .train()	src/baselines/	algorithm_registry.py	experiment .algorithm.name

All three pathways follow the identical discover-by-string-key pattern. Contributors only ever touch one pathway.

Section 3: Workflows

Init, step-loop, and training loop in detail

Initialization Flow — From YAML to Running Environment

What happens between `python run_from_config.py` and the first `env.step()` ?

① Load Configs

```
load_all_configs()  
reads 5 YAML files
```

② Build Agents

```
build_agents(cfg)  
→ list[Agent]
```

③ Create Env

```
GridWorldEnv  
(agents, **params)
```

④ Wire

```
obs_builder  
bound as callable
```

⑤ Wire Rewards

```
reward_fn closure  
bound to env
```

⑥ Start Training

```
AlgorithmClass  
(env, cfg).train()
```

Result: a fully wired environment — observations, rewards, and algorithm connected without the algorithm knowing how.

Per-Step Pipeline — Inside `env.step(actions)`

Called once per timestep. Every stage is deterministic given the same state and seed.

- | |
|---|
| ① Reset per-step trackers (<code>_captured_this_step = []</code>) |
| ② Move all agents simultaneously (based on actions dict) |
| ③ Detect captures (predator + prey on same cell → capture) |
| ④ Compute rewards (<code>reward_fn(self)</code> → dict[name → float]) |
| ⑤ Check termination / truncation (all prey captured OR max_steps reached) |
| ⑥ Render frame (only if <code>render_mode = "human"</code> or <code>"rgb_array"</code>) |
| ⑦ Build observations (<code>observation_builder(self)</code> → dict[name → dict]) |
| ⑧ Return result dict (obs, reward, terminated, truncated, info) |

■ Returns a dict — not the standard Gymnasium (obs, rew, term, trunc, info) tuple. Gymnasium wrapper is a Week-1 deliverable.

The Training Loop — IQL Episode Flow

```
for episode in range(num_episodes):
    obs = env.reset()
    done = False
    while not done:
        actions = {}
        for agent, agent_obs in obs.items():
            state = _encode_state(agent_obs)
            if random() < epsilon:
                actions[agent] = randint(0, action_dim - 1)
            else:
                actions[agent] = argmax(q_tables[agent][state])

        result = env.step(actions)

        for agent in agent_names:
            s = _encode_state(obs[agent])
            a = actions[agent]
            r = result["reward"][agent]
            s_ = _encode_state(result["obs"][agent])
            td = r + gamma * max(q_tables[agent][s_].values()) \
                - q_tables[agent][s][a]
            q_tables[agent][s][a] += alpha * td

        obs = result["obs"]
        done = result["terminated"] or result["truncated"]

    epsilon = max(min_epsilon, epsilon * epsilon_decay)
```

Epsilon-greedy

Random with prob ϵ ,
greedy otherwise.
 ϵ decays per episode.

Bellman Update

$Q(s,a) += \alpha[\gamma \cdot \max_{a'} Q(s',a') - Q(s,a) + r]$
TD error drives convergence.

defaultdict

Q-tables auto-init new
states to zero.
No pre-allocation needed.

Epsilon Decay

Exploration $\rightarrow$ exploitation.
Decays after every episode,
not every step.

Section 4: File Structure

Where everything lives and why

Annotated Directory Structure

One directory = one responsibility.

```
Predator-Prey-Archetype-Gridworld-Environment/  
src/  
  multi_agent_package/  
    core/          <- immutable physics (gridworld.py, agent.py)  
    observations/  <- 5 builders: LocalOnly LocalRadius Default Absolute Relative  
    rewards/       <- 3 functions: BaseReward PredatorDistance Survival  
    registry/      <- 3 registries: observation, reward (algorithm in baselines)  
    scripts/       <- run_from_config.py evaluate.py sweep.py render.py  
  baselines/  
    IQL/ CQL/ MIXED/ <- one folder per algorithm  
    base.py         <- BaseAlgorithm abstract class  
    registry/       <- algorithm registry  
  configs/          <- env.yaml agents.yaml observations.yaml rewards.yaml experiment*.yaml  
  tests/            <- 189 pytest tests across 9 files (all architecture layers)  
  notebooks/        <- 01_iql_demo.ipynb 02_cql_demo.ipynb  
  wiki/             <- 38 documentation files (SDD, ADRs, guides, concepts)  
  slides/           <- presentation.pptx presentation.pdf
```

Where do I...?

Change how agents see

→ [observations/](#)

Change what agents want

→ [rewards/](#)

Add new algorithm

→ [baselines/](#)

Tune experiment params

→ [configs/](#)

Understand the system

→ [wiki/](#)

Run a demo

→ [notebooks/](#)

Change movement rules

→ [talk to maintainer](#)

Key Files and Their Roles

File	Layer	Role
core/gridworld.py	L1 Core	Grid simulation, step loop, capture detection
core/agent.py	L1 Core	Agent state, movement, 5-action map
observations/*.py	L2 Plugin	Transform env state to per-agent percepts
rewards/*.py	L2 Plugin	Compute per-agent reward signals
registry/observation_registry.py	L2 Registry	Map YAML string keys to builder classes
baselines/IQL/iql.py	L3 Baseline	Independent Q-Learning — one Q-table per agent
baselines/CQL/cql.py	L3 Baseline	Centralised Q-Learning — joint action Q-tensor
baselines/MIXED/mixed.py	L3 Baseline	Per-team algorithm assignment
scripts/run_from_config.py	L4 Script	Entry point — loads configs, wires env, trains
scripts/evaluate.py	L4 Script	Greedy rollout + per-agent metrics collection

Section 5: Class & Code Structure

The key abstractions and how they connect

Core Classes — GridWorldEnv and Agent

GridWorldEnv(gym.Env)

Attributes:

`agents` list[Agent]
`_obstacle_location` set of (x,y)
`_captured_agents` set of names
`reward_fn` Callable
`observation_builder` Callable
`capture_threshold` int
`max_steps` int | `seed` int

Methods:

`reset(seed)` → obs_dict
`step(actions)` → result_dict
`base_reward()` → float
`_build_observations()`

Agent

Attributes:

`agent_name` str
`agent_type` 'predator' | 'prey'
`agent_team` str
`_agent_location` [x, y]
`is_captured` bool
`agent_speed` int (not yet wired)
`stamina` float (not yet wired)

Action Map:

0=Right 1=Up 2=Left 3=Down 4=Noop

All agents move simultaneously — not turn-by-turn.

Plugin Interfaces — Three Abstract Base Classes

ObservationBuilder

```
class ObservationBuilder:
    def __init__(self,
                  **params): ...

    def build(
        self, env
    ) -> Dict[str, dict]:
        # Read env state
        # Return per-agent
        # observation dicts
        # READ-ONLY
        ...
```

env.observation_builder =
builder_instance.build
Called each step.

RewardFunction

```
class RewardFunction:
    def __init__(self,
                  weight=1.0):
        self.weight = weight

    def compute(
        self, env
    ) -> Dict[str, float]:
        # Return per-agent
        # reward contribution
        # x self.weight
        # READ-ONLY
        ...
```

Multiple functions combined
into one closure.
Weights from YAML.

BaseAlgorithm

```
class BaseAlgorithm:
    def __init__(self,
                  env, config): ...

    def select_actions(
        self, obs: dict
    ) -> dict: # required

    def train(self):
        ... # required

    def evaluate(self,
                  episodes, max_steps):
        ... # inherited
```

Only env.reset() and
env.step() allowed.
No internal reads.

The Black-Box Algorithm Contract

What the algorithm sees — and what it must never access.

✓ Allowed

- ✓ `env.reset(seed=n)`
- ✓ `env.step(actions_dict)`
- ✓ `result['obs']`
- ✓ `result['reward']`
- ✓ `result['terminated']`
- ✓ `result['truncated']`
- ✓ `result['info']`
- ✓ `env.observation_space`
- ✓ `env.action_space`

✗ Must Never Touch

- ✗ `env.agents`
- ✗ `env._obstacle_location`
- ✗ `env._captured_agents`
- ✗ `agent._agent_location`
- ✗ `agent.is_captured`
- ✗ `env.reward_fn`
- ✗ `env.observation_builder`

The rule: any algorithm written here must work with any env that implements `reset()` + `step()`. That is what makes baselines reusable across all experimental conditions.

Section 6: Connection to RL Theory

From Bellman equations to running code

Single-Agent MDP — Every Component Maps to Code

A Markov Decision Process is (S, A, R, P, γ). Here is exactly where each element lives in this project.

MDP	Symbol	In This Codebase	Source File
State Space	S	Output of ObservationBuilder.build(env)	observations/*.py
Action Space	A	5-action discrete: {Right, Up, Left, Down, Noop}	core/agent.py
Reward Function	R(s,a)	RewardFunction.compute(env) + base_reward()	rewards/*.py + core/
Transition Model	P(s' s,a)	GridWorldEnv.step() — deterministic given seed	core/gridworld.py
Discount Factor	γ	config['experiment']['gamma'] (default 0.95)	configs/experiment.yaml
Policy	$\pi(a s)$	Q-table + epsilon-greedy in select_actions()	baselines/IQL/iql.py
Value Function	Q(s,a)	defaultdict — states init to zero on first visit	baselines/IQL/iql.py

The Bellman Equation — Math to Code

The update rule that drives all tabular Q-learning in this project.

$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot [r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a)]$$

α (alpha)

Learning rate
How fast we update
Default: 0.1

r

Immediate reward
result["reward"]
[agent_name]

γ (gamma)

Discount factor
Future reward weight
Default: 0.95

$\max Q(s', \cdot)$

Best Q-value
in next state
over all actions

$- Q(s, a)$

Current estimate
TD error =
bracketed term

```
td_error = reward + gamma * max(q_tables[agent][s_next].values()) - q_tables[agent][s][a]
q_tables[agent][s][a] += alpha * td_error
```

`q_tables` is a `defaultdict(lambda: defaultdict(float))` — new states auto-init to zero.

From MDP to Markov Game — The MARL Jump

Adding more agents changes the problem structure — not just its size.

Single-Agent MDP

State: S

Action: A (one agent)

Reward: $R(s, a)$

Trans: $P(s' | s, a)$

Goal: $\max \sum \gamma^t R_t$

Environment is stationary:

rules do not change as agent learns

Markov Game (MARL)

State: S (shared by all agents)

Action: $A^1 \times A^2 \times \dots \times A^N$ (joint)

Reward: $R^i(s, a^1, \dots, a^N)$ per agent

Trans: $P(s' | s, a^1, \dots, a^N)$

Goal: each agent $\max \sum \gamma^t R_t^i$

Non-stationarity:

from agent i 's view, 'the environment'

keeps changing as others learn

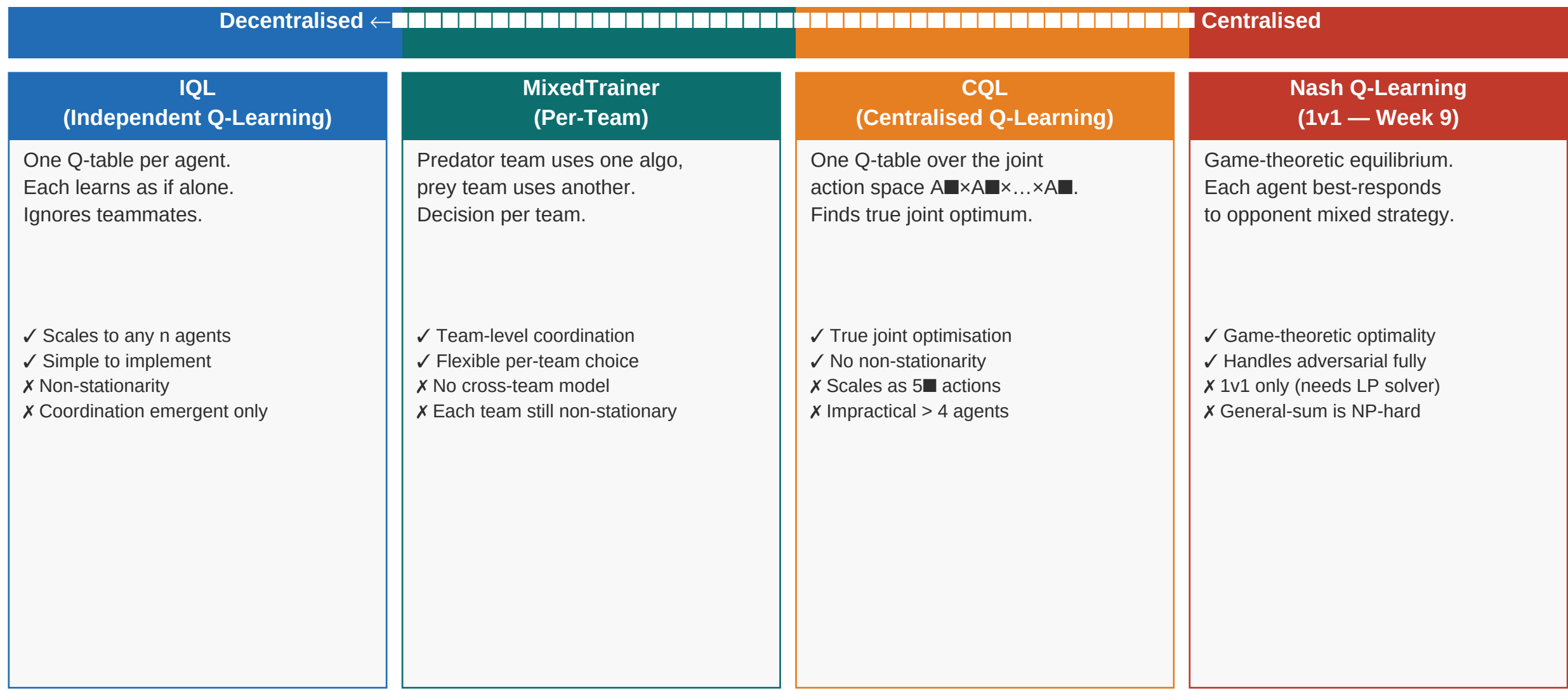
In this env: predators and prey are adversarial teams. Each team's reward depends on the other's actions. Neither side is stationary from the other's perspective.

Section 7: The MARL Algorithm Spectrum

How the architecture enables different coordination strategies

The Algorithm Spectrum — Decentralised to Centralised

Each algorithm trades coordination ability against scalability.



Why the Architecture Makes Extensibility Effortless

Adding CQL, MixedTrainer, and Nash Q each required: one file, one registry line, one YAML. Zero core changes.

Adding CQL to an IQL-only repo

Before: Only IQL.py in baselines/



After: + CQL.py in baselines/CQL/
+ one line in algorithm_registry.py
+ experiment_cql.yaml in configs/
= zero changes to core, obs, rewards, or IQL

Adding MixedTrainer

Before: Only IQL.py + CQL.py



After: + mixed.py in baselines/MIXED/
+ one line in algorithm_registry.py
+ experiment_mixed.yaml
= zero changes anywhere else

Adding Nash Q-Learning (Week 9)

Before: IQL + CQL + MixedTrainer existing



After: + nashq.py in baselines/NASHQ/
+ one line in algorithm_registry.py
+ experiment_nashq.yaml
= zero changes to prior baselines or core

Invariant: adding an algorithm never changes the environment, plugins, or other algorithms. Scientific comparability is always preserved.

The Scaling Wall — When CQL Breaks Down

Joint action spaces grow exponentially. This is why deep MARL methods exist.

Setup	Agents (n)	IQL Q-tables	CQL Joint Actions	CQL Practical?
1v1	2	$2 \times S $	$5^2 = 25$	Yes
1v2	3	$3 \times S $	$5^3 = 125$	Yes
2v2	4	$4 \times S $	$5^4 = 625$	Yes (barely)
3v3	6	$6 \times S $	$5^6 = 15,625$	No
5v5	10	$10 \times S $	$5^{10} \approx 10M$	Never

Joint space grows as:


$$d^n$$

$d = 5 \text{ actions} \cdot n = \text{agents}$

Why IQL survives:

Each agent has its OWN
Q-table — no joint space.
 $O(n \cdot |S| \cdot d)$ not $O(|S| \cdot d^n)$

This project: optimised for ≤ 4 agents on grids $\leq 10 \times 10$. For larger games, the natural next step is CTDE (Centralised Training, Decentralised Execution) methods such as QMIX or MADDPG — deep RL baselines planned for future work.

Section 8: Contributing

Rules, pathways, and checklist

The Golden Rule — Immutable Core

One rule protects the scientific integrity of every experiment run on this codebase.

core/gridworld.py and core/agent.py are NEVER modified by contributors.

All variation is expressed through config files, observation plugins, reward plugins, and algorithm plugins.

Why?

- Any physics change invalidates comparisons across ALL prior experiments — you lose the baseline
- The invariant is what makes this a fair test bed: change the rules, lose the comparison
- Every variation you need can be expressed through the three plugin layers
- If it cannot, that is a discussion for maintainers — not a unilateral edit
- Enforced by: code review + CONTRIBUTING.md + ADR-001

The Three Contribution Pathways

Every contribution fits one of these patterns. Following the pathway guarantees a clean merge.

Pathway 1: New Observation Builder

1. `src/observations/my_obs.py` – subclass `ObservationBuilder`, implement `build(env)`
2. Add one line to `observation_registry.py`
3. Add YAML key to `configs/observations.yaml`
4. Write tests in `tests/test_observations.py`

Pathway 2: New Reward Function

1. `src/rewards/my_reward.py` – subclass `RewardFunction`, implement `compute(env)`
2. Add one line to `reward_registry.py`
3. Add YAML block to `configs/rewards.yaml`
4. Write tests in `tests/test_rewards.py`

Pathway 3: New Learning Algorithm

1. `src/baselines/MYALGO/myalgo.py` – subclass `BaseAlgorithm`, implement `select_actions()` + `train()`
2. Add self-registration guard at module level
3. Import in `baselines/__init__.py`
4. Add `configs/experiment_myalgo.yaml`
5. Write tests in `tests/test_baselines_myalgo.py`

Section 9: End-to-End Example

One command, traceable from YAML to trained policy

The Scenario — Train Predators to Capture Prey

Goal: 2 predators learn to capture 2 prey on a 6×6 grid using IQL + distance shaping.

```
# env.yaml
grid_size: 6
max_steps: 200
capture_threshold: 1
perc_num_obstacle: 15.0
render_mode: null
seed: 42
```

```
# agents.yaml
agents:
  - name: pred_0
    type: predator
    team: predators
  - name: pred_1
    type: predator
    team: predators
  - name: prey_0
    type: prey
    team: prey
  - name: prey_1
    type: prey
    team: prey
```

```
# observations.yaml
observation:
  type: local_radius
  params:
    radius: 3

# rewards.yaml
rewards:
  - name: base_reward
    weight: 1.0
  - name: predator_distance
    weight: 0.5
```

ONE COMMAND

```
python -m multi_agent_package.scripts.run_from_config --config configs/
```

```
# experiment.yaml
experiment:
  algorithm:
    name: iql
  episodes: 1000
  alpha: 0.1
  gamma: 0.95
  epsilon: 1.0
  epsilon_decay: 0.995
  min_epsilon: 0.01
```

The only code you write is config. The system assembles everything.

One Command — Full Training Run

What happens when you hit Enter:

```
python -m multi_agent_package.scripts.run_from_config --config configs/
```

① Load	② Agents	③ Env	④ Obs	⑤ Reward	⑥ Train
5 YAML files → merged config	4 Agent objects pred_0...prey_1	GridWorldEnv size=6, seed=42	LocalRadius(r=3) → env.obs_builder	Base + Distance → env.reward_fn	IQL.train() 1000 episodes

Console output:

```
Episode 100/1000 | avg_reward: -12.4 | captures: 0.12 | epsilon: 0.607
```

```
Episode 200/1000 | avg_reward: -8.1 | captures: 0.31 | epsilon: 0.368
```

```
Episode 500/1000 | avg_reward: 2.7 | captures: 0.64 | epsilon: 0.082
```

```
Episode 1000/1000 | avg_reward: 14.2 | captures: 0.88 | epsilon: 0.010
```

```
Training complete. Saved: checkpoints/iql_pred_0.pkl (+ 3 others)
```

What Success Looks Like

Same seed + same YAMLS reproduces this result exactly on any machine.

Metric	Trained Policy	Random Policy
Capture rate	88%	12%
Avg episode length	67 steps	198 steps
Avg total reward (predators)	+14.2	-8.6
Avg total reward (prey)	-31.0	-4.1
Unique Q-states visited	1,847	312

```
python -m multi_agent_package.scripts.evaluate \
  --checkpoint checkpoints/ --episodes 200
```

How to read the results

■ Reward curve rising

Predators learned to move toward prey.
Distance shaping drove this behaviour.

■ Episode length falling

Captures happen earlier as policy improves.
Random agents always hit max_steps.

■ States visited growing

Q-table coverage expands during exploration
then stabilises when epsilon is low.

■ Prey reward drops too

Prey Q-tables also learn evasion —
but predators win this game.

Where We Are — and Where We're Going

Ten weeks, four students, one JOSS paper.

Current State (start of Week 1)

- ✓ Discrete gridworld (up to 10×10)
- ✓ 5 observation builders
- ✓ 3 reward functions
- ✓ IQL + CQL + MixedTrainer (tabular)
- ✓ 189 pytest tests across 9 files
- ✓ Full SDD wiki (38 documents)
- ✓ Interactive notebooks (IQL + CQL)
- ✓ CI pipeline (in progress)

10-Week Roadmap

Week 1 Bug fixes • Gymnasium wrapper • CI passing

Weeks 2–4 IDQN — neural Q-learning per agent

Weeks 5–6 IPPO — actor-critic, policy gradient

Weeks 7–9 Nash Q-Learning — 1v1 minimax

Week 10 JOSS paper + final documentation

Resources & Next Steps

Start Here

- [wiki/students/index.md](#) — curated student documentation
- [notebooks/01_iql_demo.ipynb](#) — interactive IQL walk-through
- [guides/quickstart.md](#) — install + first run in 5 minutes

Key References

- Sutton & Barto (2018) — Chapters 6, 13
- Hu & Wellman (2003) — Nash Q-Learning
- Littman (2001) — Friend-or-Foe Q-Learning

This Week

- [wiki/weekly/week-01.md](#) — task list with owners
- [wiki/reviews/audit-2026-06-07.md](#) — known bugs
- [CONTRIBUTING.md](#) — pull request checklist

Build understanding from the ground up.